

백엔드 프론트 구조 (FastAPI + SQLAlchemy + SQLite)

Plan 모드에 입력 → 질문에 답하며 계획 다듬기 → "구현 시작"으로 Agent 실행. 순수 질문은 Ask 모드.

아래 프롬프트에서 (여기에 적으세요) 라고 된 부분만 자유롭게 채우세요. 나머지 줄은 그대로 복사해서 쓰면 됩니다.

작업 순서

1. [1단계: 프로젝트 구조] 프롬프트를 **Plan 모드**에 입력합니다.
2. 질문에 답하고 "**구현 시작**" 으로 Agent가 기본 구조를 만들게 합니다.
3. 기능을 하나 정합니다.
4. [2단계: DB 테이블 생성] 프롬프트를 Plan 모드에 입력하고 같은 방식으로 진행합니다.
5. [3단계: API 만들기] 프롬프트를 Plan 모드에 입력하고 같은 방식으로 반복합니다.
6. `/docs` (Swagger)에서 직접 눌러 확인합니다.
7. 확정된 응답 형태를 프론트 프롬프트에 복사합니다.
8. 순수 질문(용어, 에러 의미 등)은 **Ask 모드**로.

1단계: FastAPI 프로젝트 구조 만들기

💡 아래 프롬프트는 Plan 모드에 입력하세요. 질문에 답하며 다듬은 뒤 "구현 시작"을 누르면 Agent가 실제로 프로젝트를 만듭니다.

구조

[지금 상황]

- (여기에 적으세요: 프로젝트가 있는지 없는지)

[부탁하고 싶은 것]

기본 프로젝트 구조를 만들어줘.

[지켜야 할 것]

- **app** 폴더 하나로 모으고, 그 안에 서버 시작 파일, DB 연결 파일, 테이블 정의 파일, **API** 코드를 담을 폴더로 나눠줘.
- 이걸 실행할 수 있도록 가상환경을 만들고, 필요한 패키지(**FastAPI**, **SQLAlchemy**)를 설치해줘.
- 가상환경 활성화 방법, 설치 명령어, 서버 실행 명령어, `/docs` 확인 방법을 **README.md**에 정리해줘.
- 지금 당장 채우지 않고 다음 단계(테이블 정의, **API** 라우트)에서 채울 파일에는 **TODO**: 무엇을 해야 하는지 - 왜 필요한지 로 표시해줘. 지금 바로 동작해야 하는 파일(서버 시작 파일, DB 연결 파일)에는 **TODO**를 넣지 마.

[확인 방법]

- **README.md**에 적힌 대로 그대로 따라 했을 때, 에러 없이 서버가 뜨고 `/docs` 화면이 보이는지 확인해줘.

예시

[지금 상황]

- 프로젝트 자체가 아직 없다. 빈 폴더만 있는 상태다.

[부탁하고 싶은 것]

할 일 관리 서버의 기본 프로젝트 구조를 만들어줘.

[지켜야 할 것]

- **app** 폴더 하나로 모으고, 그 안에 서버 시작 파일, **DB** 연결 파일, 테이블 정의 파일, **API** 코드를 담을 폴더로 나눠줘.
- 이걸 실행할 수 있도록 가상환경을 만들고, 필요한 패키지(**FastAPI**, **SQLAlchemy**)를 설치해줘.
- 가상환경 활성화 방법, 설치 명령어, 서버 실행 명령어, **/docs** 확인 방법을 **README.md**에 정리해줘.
- 지금 당장 채우지 않고 다음 단계(테이블 정의, **API** 라우트)에서 채울 파일에는 **TODO**: 무엇을 해야 하는지 - 왜 필요한지 로 표시해줘. 지금 바로 동작해야 하는 파일(서버 시작 파일, **DB** 연결 파일)에는 **TODO**를 넣지 마.

[확인 방법]

- **README.md**에 적힌 대로 그대로 따라 했을 때, 에러 없이 서버가 뜨고 **/docs** 화면이 보이는지 확인해줘.

2단계: DB 테이블 생성

💡 아래 프롬프트도 Plan 모드에 입력하세요.

구조

[저장할 데이터]

- (여기에 적으세요: 무엇에 대한 정보인지)
- (여기에 적으세요: 저장할 정보 목록. 예: 제목, 완료 여부, 만든 날짜)
- (여기에 적으세요: 연결되는 다른 데이터가 있는지)

[지금 상황]

- 프로젝트 구조는 이미 있음 (1단계에서 생성)
- (여기에 적으세요: 관련 기존 코드가 있으면 붙여넣기)

[부탁하고 싶은 것]

이 데이터를 저장할 수 있는 테이블 구조를 만들고, 실제로 데이터베이스에 테이블을 생성해줘.

[지켜야 할 것]

- 데이터 형식과 저장 규칙은 내가 알아서 적절하게 정하고, 왜 그렇게 정했는지 쉬운 말로 설명해줘.
- **SQLite** 파일 하나로 저장해줘.
- 미완성 부분은 **TODO**: 무엇을 해야 하는지 - 왜 필요한지 로 남겨줘.

[확인 방법]

- 서버를 실행했을 때 에러 없이 테이블이 만들어지는지 확인해줘.
- (여기에 적으세요: 파일을 어떻게 나눠서 받고 싶은지, 없으면 "네가 적절히 나눠줘")

예시

[저장할 데이터]

- 할 일(Todo) 하나하나에 대한 정보
- 제목, 설명(없어도 됨), 완료 여부, 만든 날짜
- 연결되는 데이터 없음. 단순하게 관리한다.

[지금 상황]

- 프로젝트 구조는 이미 있음 (1단계에서 생성)

[부탁하고 싶은 것]

이 데이터를 저장할 수 있는 테이블 구조를 만들고, 실제로 데이터베이스에 테이블을 생성해줘.

[지켜야 할 것]

- 데이터 형식과 저장 규칙은 네가 알아서 적절하게 정하고, 왜 그렇게 정했는지 쉬운 말로 설명해줘.
- SQLite 파일 하나로 저장해줘.
- 미완성 부분은 **TODO**: 무엇을 해야 하는지 - 왜 필요한지 로 남겨줘.

[확인 방법]

- 서버를 실행했을 때 에러 없이 테이블이 만들어지는지 확인해줘.
- 데이터 구조를 정의하는 파일 하나로 줘.

3단계: API 만들기 (기능 하나씩 반복)

💡 아래 프롬프트도 Plan 모드에 입력하세요.

구조

[지금 상황]

- 테이블은 이미 만들어져 있음 (2단계에서 생성)
- (여기에 적으세요: 관련 기존 코드가 있으면 붙여넣기)

[부탁하고 싶은 것]

- (여기에 적으세요: 어떤 기능이 필요한지. 예: 추가/조회/수정/삭제 등)

[지켜야 할 것]

- 존재하지 않는 데이터를 요청하면 에러를 알려줘.
- 새 외부 라이브러리는 임의로 추가하지 마.
- 미완성 부분은 **TODO**: 무엇을 해야 하는지 - 왜 필요한지 로 남겨줘.

[확인 방법]

- 화면(Swagger)에서 직접 눌러서 확인할 수 있게 해줘.
- (여기에 적으세요: 파일을 어떻게 나눠서 받고 싶은지, 없으면 "네가 적절히 나눠줘")

예시

[지금 상황]

- todos 테이블은 이미 만들어져 있음 (2단계에서 생성).

[부탁하고 싶은 것]

할 일을 관리하는 **API**를 만들어줘.

- 추가 / 목록 조회 / 상세 조회 / 수정 / 삭제 / 완료 표시 켜고 끄기

[지켜야 할 것]

- 존재하지 않는 할 일 조회/수정/삭제 시 에러를 알려줘.
- 새 외부 라이브러리는 임의로 추가하지 마.
- 미완성 부분은 **TODO**: 무엇을-왜 형태로 남겨줘.

[확인 방법]

- `/docs` 화면에서 6개 기능을 직접 눌러서 확인한다.
- **API** 코드 파일 하나로 줘.

막힐 때: Ask 모드

Plan이나 Agent가 만든 코드를 고치려는 게 아니라, 그냥 궁금한 게 생겼을 때 쓰는 모드입니다. Ask 모드는 파일을 건드리지 않고 설명만 해주기 때문에, 지금 진행 중인 계획이나 코드에 실수로 영향을 주지 않습니다.

이럴 때 쓰면 됩니다.

- 코드에 나온 낯선 용어나 문법이 궁금할 때
- 에러 메시지가 무슨 뜻인지 궁금할 때
- 왜 이렇게 만들었는지 이유가 궁금할 때

방금 만든 코드에서 (여기에 적으세요: 궁금한 부분)이 정확히 뭘 의미하는지 쉽게 설명해줘.

이 에러 메시지가 무슨 뜻인지 쉽게 설명해줘: (여기에 적으세요: 에러 메시지 붙여넣기)

질문이 끝나면 다시 Plan 모드로 돌아가서 하던 작업을 이어가면 됩니다.

테이블 구조를 나중에 더 다듬고 싶을 때

이미 만든 테이블에 항목을 추가하거나 구조를 조정하고 싶을 때 쓰는 프롬프트입니다.

[스키마 조정 요청]

지금 있는 테이블에 이런 내용을 추가/변경하고 싶어: (여기에 적으세요)

지켜야 할 것:

- 데이터 형식과 저장 규칙은 내가 알아서 적절하게 정하고, 왜 그렇게 정했는지 쉬운 말로 설명해줘.
- 기존에 저장된 데이터가 사라지지 않도록 주의해줘.